



Computer Science & IT



# Data Structure & programming

Graph & Hashing

Lecture No. 01



By- Abhishek Sir

# Recap of Previous Lecture



Topic

Minimum No. of Nodes in AVL tree

Topic

Heap counting

Topic

BST counting

Topic

Topic



# Topics to be Covered



Topic

Hashing

Topic

chaining

Topic

Linear probing

Topic

Topic



## Topic : Hashing



44 marks  $\rightarrow$  65 - 70



Every mistake  
pattern identity  
Weekness





## Topic : Hashing



Searching

Direct Addressing  
 $O(1)$

Array  $\left\{ \begin{array}{l} \text{ordered} \quad \therefore O(\log_2 n) \\ \text{unordered} \quad \therefore O(n) \end{array} \right.$

Linked list  $\left\{ \begin{array}{l} \text{ordered} \quad \therefore O(n) \\ \text{unordered} \quad \therefore O(n) \end{array} \right.$  No Random Access

BST  $\left\{ \begin{array}{l} \text{Balanced (AVL)} \quad \therefore O(\log_2 n) \\ \text{BST} \quad O(n) \end{array} \right.$

Tree :  $\longrightarrow$  Traversal :  $O(n)$



## Topic : Hashing



Direct Addressing: In direct Addressing a Large array is used and Index of array represents the key value.

$k = 128$  will be available at  $A[k]$

Time complexity for search will be  $O(1)$





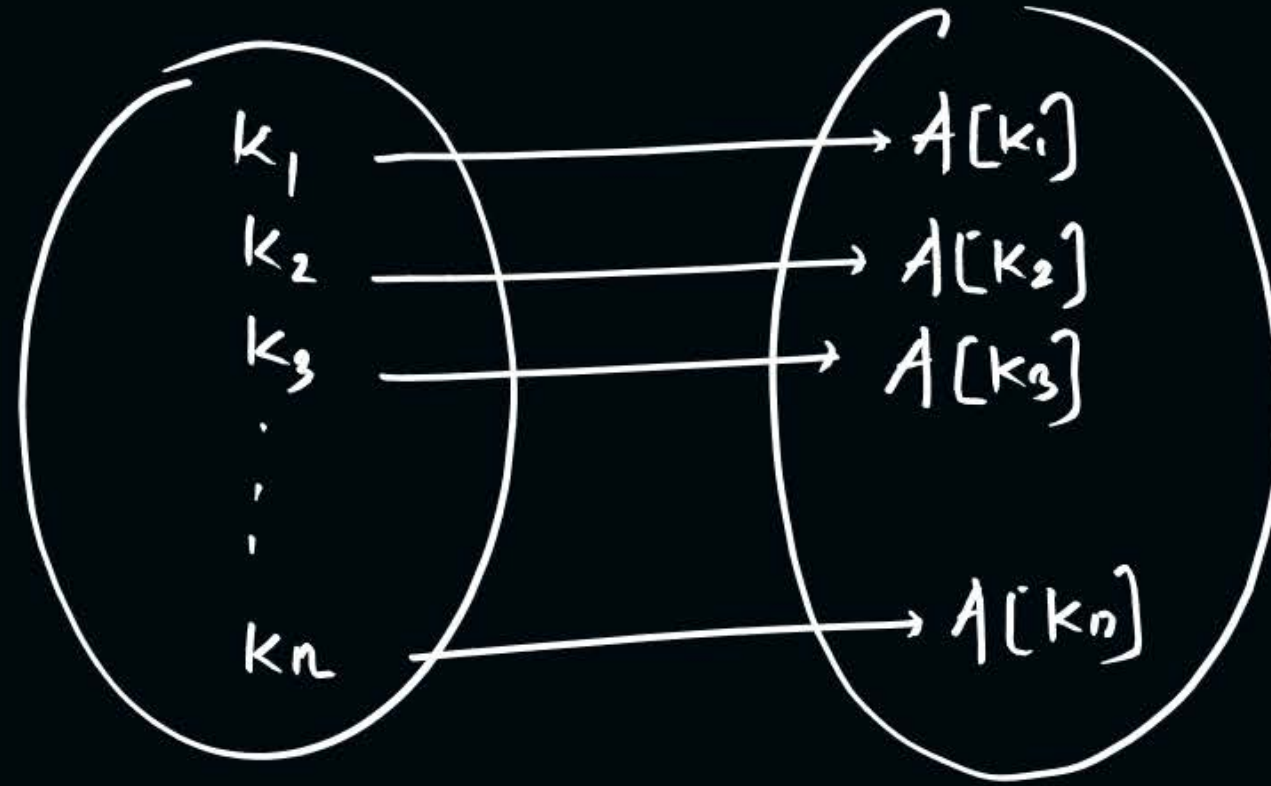
## Topic : Hashing



Disadvantage : A Large size of array required

(II) wastage of space, Since all Index may not represent the key value.

Suboptimal solution is rather than using large size of array lets use small size array (Hash table) and to distribute the key use a function (Hash function)



direct Addressing

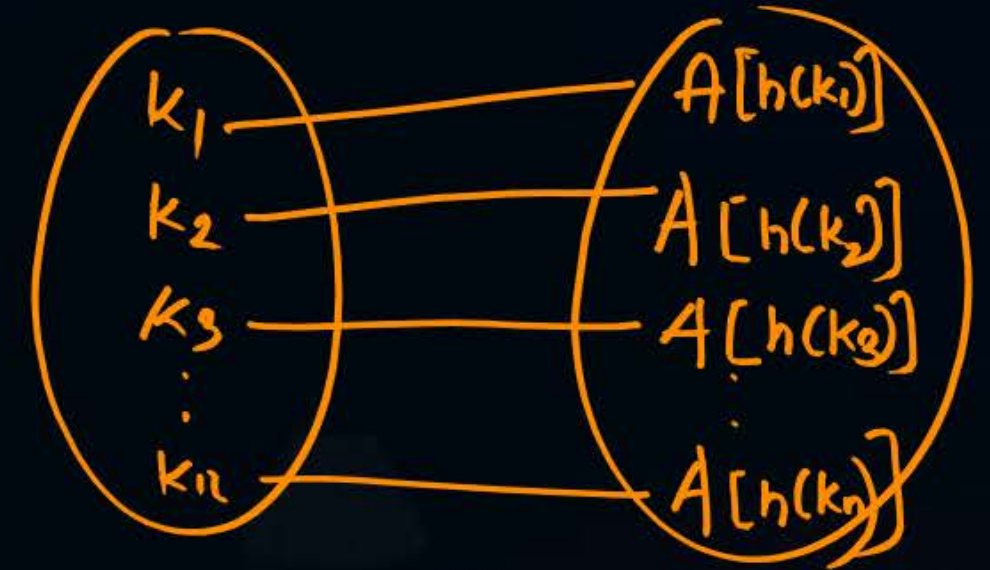
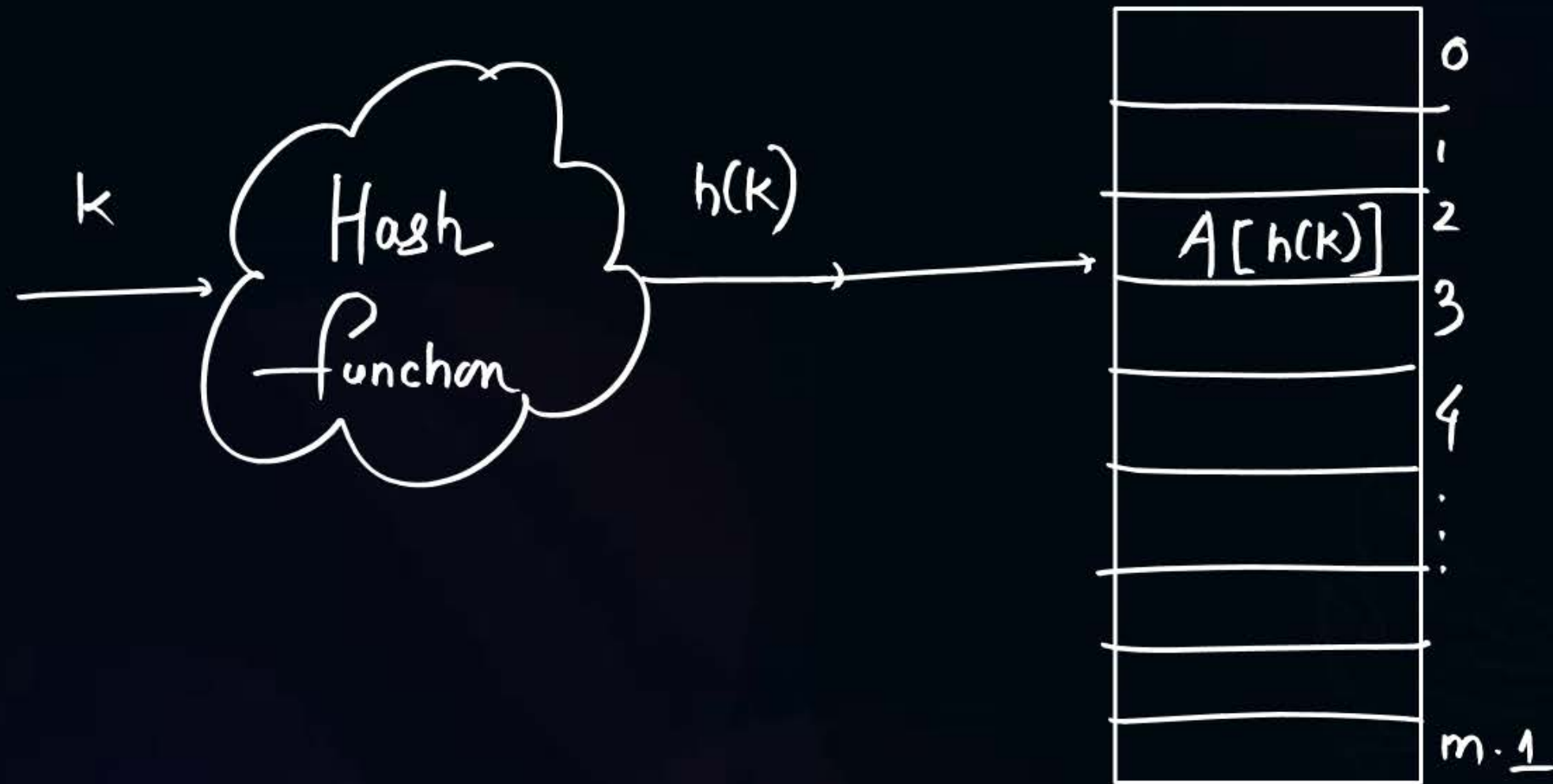




## Topic : Hashing



$k \rightarrow A[k]$





## Topic : Hashing



I. Hash function must be able to map any key value to Index of table

II. The distribution should be uniform





## Topic : Hashing



### Terminology

1.  $k$  is key value
2.  $T$  is hash-table
3.  $h(k)$  is hash-function
4.  $m$ . Size of table Index from 0 to  $m-1$
5.  $n$  No. of keys to mapped.

Each Location of  
table is called  
Bucket or slot.



## Topic : Hashing



Which one of the following hash functions on integers will distribute keys most uniformly over 10 buckets numbered 0 to 9?

(A)  $h(i) = i^2 \bmod 10$

*Non uniform distribution*

(B)  $h(i) = i^3 \bmod 10$

(C)  $h(i) = (11 * i^2) \bmod 10$

(D)  $h(i) = (12 * i^2) \bmod 10$

*Homework*





## Topic : Hashing

|   |       |
|---|-------|
| 0 | 0,10  |
| 1 | 1,1,9 |
| 2 |       |
| 3 |       |
| 4 | 2,8   |
| 5 | 5     |
| 6 | 4,6   |
| 7 |       |
| 8 |       |
| 9 | 3,7   |

Non uniform distribution

$$h(i) = i^2 \bmod 10$$

$$h(0) = 0^2 \bmod 10 = 0$$

$$h(1) = 1^2 \bmod 10 = 1$$

$$h(2) = 2^2 \bmod 10 = 4$$

$$h(3) = 3^2 \bmod 10 = 9$$

$$h(4) = 4^2 \bmod 10 = 6$$

$$h(5) = 5^2 \bmod 10 = 5$$

$$h(6) = 6^2 \bmod 10 = 6$$

$$h(7) = 7^2 \bmod 10 = 9$$

$$h(8) = 8^2 \bmod 10 = 4$$

$$h(9) = 9^2 \bmod 10 = 1$$

$$h(10) = 10^2 \bmod 10 = 0$$

$$h(11) = 11^2 \bmod 10 = 1$$



## Topic : Hashing



|   |       |
|---|-------|
| 0 | 0, 10 |
| 1 | 1, 11 |
| 2 | 8     |
| 3 | 7     |
| 4 | 4     |
| 5 | 5     |
| 6 | 6     |
| 7 | 3     |
| 8 | 2     |
| 9 | 9     |

uniform distribution

$$h(i) = i^3 \% 10$$

$$h(0) = 0^3 \% 10 \rightarrow 0$$

$$h(1) = 1^3 \% 10 \rightarrow 1$$

$$h(2) = 2^3 \% 10 \rightarrow 8$$

$$h(3) = 3^3 \% 10 \rightarrow 7$$

$$h(4) = 4^3 \% 10 \rightarrow 4$$

$$h(5) = 5^3 \% 10 \rightarrow 5$$

$$h(6) = 6^3 \% 10 \rightarrow 6$$

$$h(7) = 7^3 \% 10 \rightarrow 3$$

$$h(8) = 8^3 \% 10 \rightarrow 2$$

$$h(9) = 9^3 \% 10 \rightarrow 9$$

$$h(10) = 10^3 \% 10 \rightarrow 0$$





## Topic : Hashing



**Collision:** Two keys mapped to same location leads to collision

$$h(k) = k^2 \bmod 10$$

$$k=4, \quad 4^2 \bmod 10 \rightarrow 6$$

$$k=6 \quad 6^2 \bmod 10 \rightarrow 6$$



## Topic : Hashing



Collision Resolution Technique

1. Open Hashing

2. Closed Hashing / Open Addressing





## Topic : Hashing



**Open Hashing** :- When collided keys stored outside of table by allocating storage.

Open Hashing used when No. of keys not known in advance

Dynamic Situation

1. Chaining



## Topic : Hashing

Chaining : If collision occurs then a new Linked list node is created and added to the list.

## Example

Let us see the following example to get better idea. If we have some elements like  $\{15, 47, 23, 34, 85, 97, 65, 89, 70\}$ . And our hash function is  $h(x) = x \bmod 7$ .



Hash table  
as array of  
head pointers





## Topic : Hashing



Load factor: Its a ratio between No. of keys to  
No. of slot.

$$\alpha = \frac{n}{m}$$



## Topic : Hashing



#Q Given a has table  $T$  with 25 slots that stores 2000 elements, the load factor  $\alpha$  for  $T$  is \_\_\_\_\_.

$$\alpha = \frac{2000}{25}$$

$$\boxed{\alpha = 80}$$



## Topic : Hashing



$$h(x) = \underline{x \% 9} \quad \text{Max: } \underline{3}$$

$$\text{Min: } \underline{0}$$

#Q. Consider a hash table with 9 slots. The hash function is, The collisions are resolved by chaining.

$$\frac{9}{9} = \underline{1}$$

The following 9 keys are inserted in the order: 5, 28, 19, 15, 20, 33, 12, 17, 10.

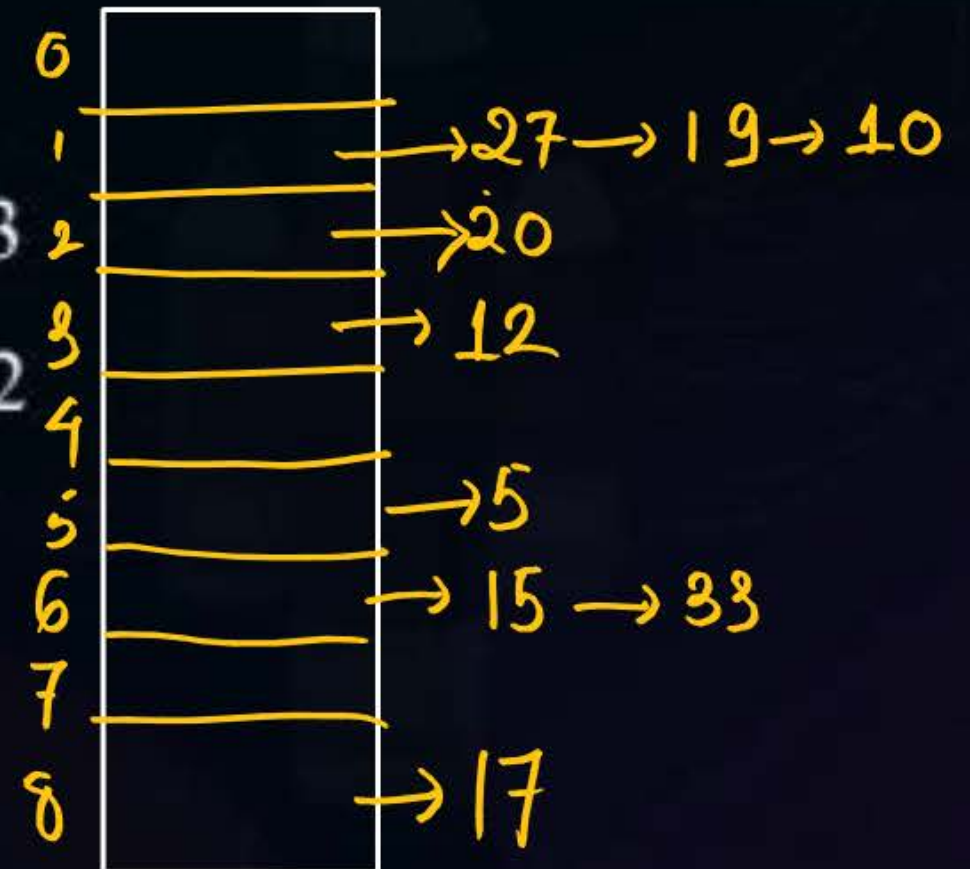
The maximum, minimum, and average chain lengths in the hash table, respectively, are

(A) 3, 0, and 1 ✓

(C) 4, 0, and 1

(B) 3, 3, and 3

(D) 3, 0, and 2







## Topic : Hashing



Q. Consider a hash table with 100 slots. Collisions are resolved using chaining. Assuming simple uniform hashing, what is the probability that the first 3 slots are unfilled after the first three insertion?

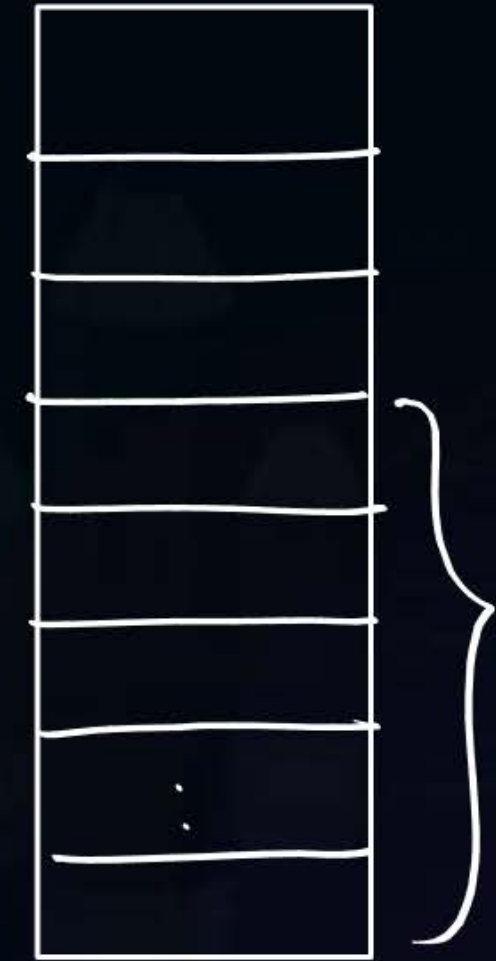
- ✓ (A)  $(97 \times 97 \times 97)/100^3$  (B)  $(99 \times 98 \times 97)/100^3$   
(C)  $(97 \times 96 \times 95)/100^3$  (D)  $(97 \times 96 \times 95)/(3! \times 100^3)$

3 slot - empty : Rest 97

First Insertion:  $\frac{97}{100}$

Third insertion:  $\frac{97}{100}$

Sec Second insertion:  $\frac{97}{100}$





## Topic : Hashing



Closed Hashing/open Addressing

In closed Hashing collided key stored inside the table in different Location.

To find different Location

1. Linear probing
2. Quadratic probing
3. Double Hashing





## Topic : Hashing



Linear probing :  $probe_0 = h(k)$

$$probe_1 = (h(k) + 1) \% m$$

$$probe_2 = (h(k) + 2) \% m$$

⋮

while (Table[probe] occupied)

$$probe = (probe + 1) \% m$$

Linearly Search for  
next empty Location





## Linear Probing Example

Consider the the hash table of size 13, the hash table is initially empty and hash  $h(k) = k \bmod 13$  is used. Collision is resolved by *linear probing*.

Insert keys:

18, 41, 22, 44, 59, 32, 31, 73  
in to hash table and show the resultant hash table.



## Linear Probing Example

18, 41, 22, 44, 59, 32, 31, 73



Linear probing

| Key | $h(x) = x \bmod 13$ |
|-----|---------------------|
| 18  | 5                   |
| 41  | $41 \% 13 = 2$      |
| 22  | $22 \% 13 = 9$      |
| 44  | $44 \% 13 = 5$      |
| 59  | $59 \% 13 = 7$      |
| 32  | $32 \% 13 = 6$      |
| 31  | $31 \% 13 = 5$      |
| 73  | $73 \% 13 = 8$      |

|    |    |
|----|----|
|    | 0  |
|    | 1  |
| 41 | 2  |
|    | 3  |
|    | 4  |
| 18 | 5  |
| 44 | 6  |
| 59 | 7  |
| 32 | 8  |
| 22 | 9  |
| 31 | 10 |
| 73 | 11 |
|    | 12 |





## Topic : Hashing



A hash table contains 10 buckets and uses linear probing to resolve collisions. The key values are integers and the hash function used is  $\text{key} \% 10$ . If the values 43, 165, 62, 123, 142 are inserted in the table, in what location would the key value 142 be inserted?

- (A) 2
- (B) 3
- (C) 4
- (D) 6

|   |     |
|---|-----|
| 0 |     |
| 1 |     |
| 2 | 62  |
| 3 | 43  |
| 4 | 123 |
| 5 | 165 |
| 6 | 142 |
| 7 |     |
| 8 |     |
| 9 |     |







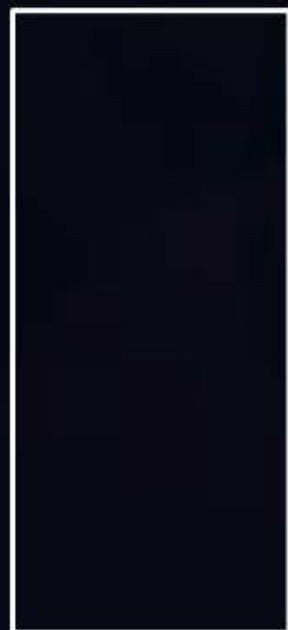
## Topic : Hashing

A hash table of length 10 uses open addressing with hash function  $h(k) = k \bmod 10$ , and linear probing. After inserting 6 values into an empty hash table, the table is as shown below

|   |    |
|---|----|
| 0 |    |
| 1 |    |
| 2 | 42 |
| 3 | 23 |
| 4 | 34 |
| 5 | 52 |
| 6 | 46 |
| 7 | 33 |
| 8 |    |
| 9 |    |

Which one of the following choices gives a possible order in which the key values could have been inserted in the table?

- A) 46, 42, 34, 52, 23, 33
- B) 34, 42, 23, 52, 33, 46
- C) 46, 34, 42, 23, 52, 33
- D) 42, 46, 33, 23, 34, 52





## Topic : Hashing



A) 46, 42, 34, 52, 23, 33

B) 34, 42, 23, 52, 33, 46

C) 46, 34, 42, 23, 52, 33

D) 42, 46, 33, 23, 34, 52

|   |    |
|---|----|
| 0 |    |
| 1 |    |
| 2 | 42 |
| 3 | 23 |
| 4 | 34 |
| 5 | 52 |
| 6 | 46 |
| 7 | 33 |
| 8 |    |
| 9 |    |

|   |    |
|---|----|
| 0 |    |
| 1 |    |
| 2 | 42 |
| 3 | 52 |
| 4 | 34 |
| 5 |    |
| 6 | 46 |
| 7 |    |
| 8 |    |
| 9 |    |

(A)

✓

X

✓

✓





## Topic : Hashing



A) 46, 42, 34, 52, 23, 33

B) 34, 42, 23, 52, 33, 46

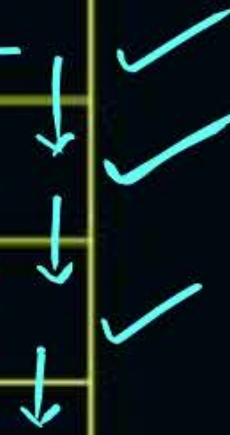
C) 46, 34, 42, 23, 52, 33

D) 42, 46, 33, 23, 34, 52

|   |    |
|---|----|
| 0 |    |
| 1 |    |
| 2 | 42 |
| 3 | 23 |
| 4 | 34 |
| 5 | 52 |
| 6 | 46 |
| 7 | 33 |
| 8 |    |
| 9 |    |



|   |    |
|---|----|
| 0 |    |
| 1 |    |
| 2 | 42 |
| 3 | 23 |
| 4 | 34 |
| 5 | 52 |
| 6 | 33 |
| 7 |    |
| 8 |    |
| 9 |    |





## Topic : Hashing



A) 46, 42, 34, 52, 23, 33

B) 34, 42, 23, 52, 33, 46

C) 46, 34, 42, 23, 52, 33 ✓

D) 42, 46, 33, 23, 34, 52

|   |    |
|---|----|
| 0 |    |
| 1 |    |
| 2 | 42 |
| 3 | 23 |
| 4 | 34 |
| 5 | 52 |
| 6 | 46 |
| 7 | 33 |
| 8 |    |
| 9 |    |

|   |    |
|---|----|
| 0 |    |
| 1 |    |
| 2 | 42 |
| 3 | 23 |
| 4 | 34 |
| 5 | 52 |
| 6 | 46 |
| 7 | 33 |
| 8 |    |
| 9 |    |



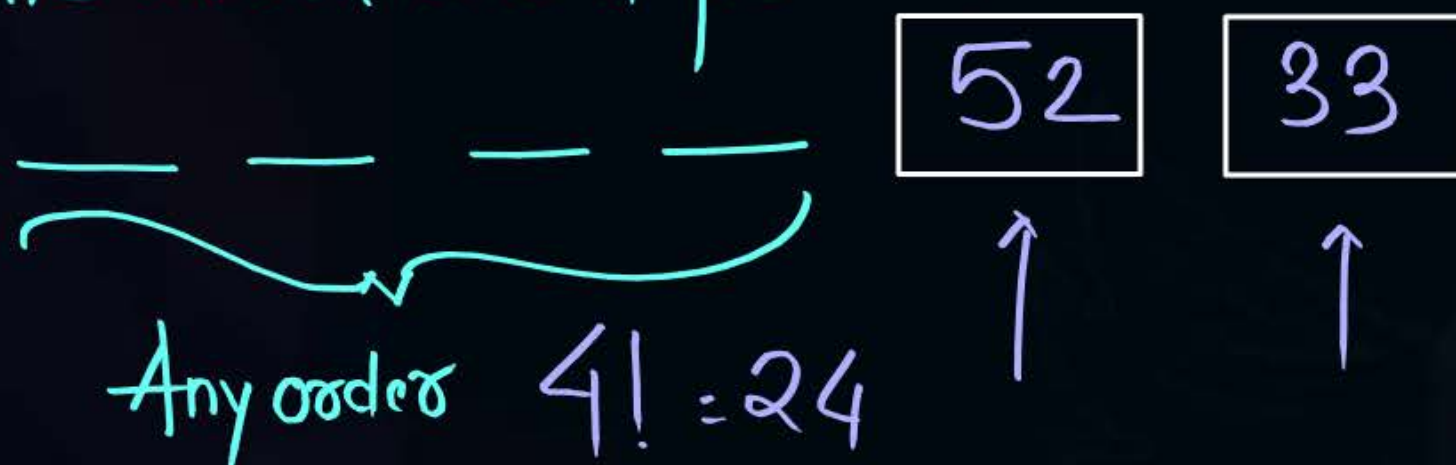


## Topic : Hashing

How many different insertion sequences of the key values using the same hash function and linear probing will result in the hash table shown above?

- (A) 10
- (B) 20
- (C) 30
- (D) 40

Counting  
4 elements are in correct position



|   |    |
|---|----|
| 0 |    |
| 1 |    |
| 2 | 42 |
| 3 | 23 |
| 4 | 34 |
| 5 |    |
| 6 | 46 |
| 7 |    |
| 8 |    |
| 9 |    |

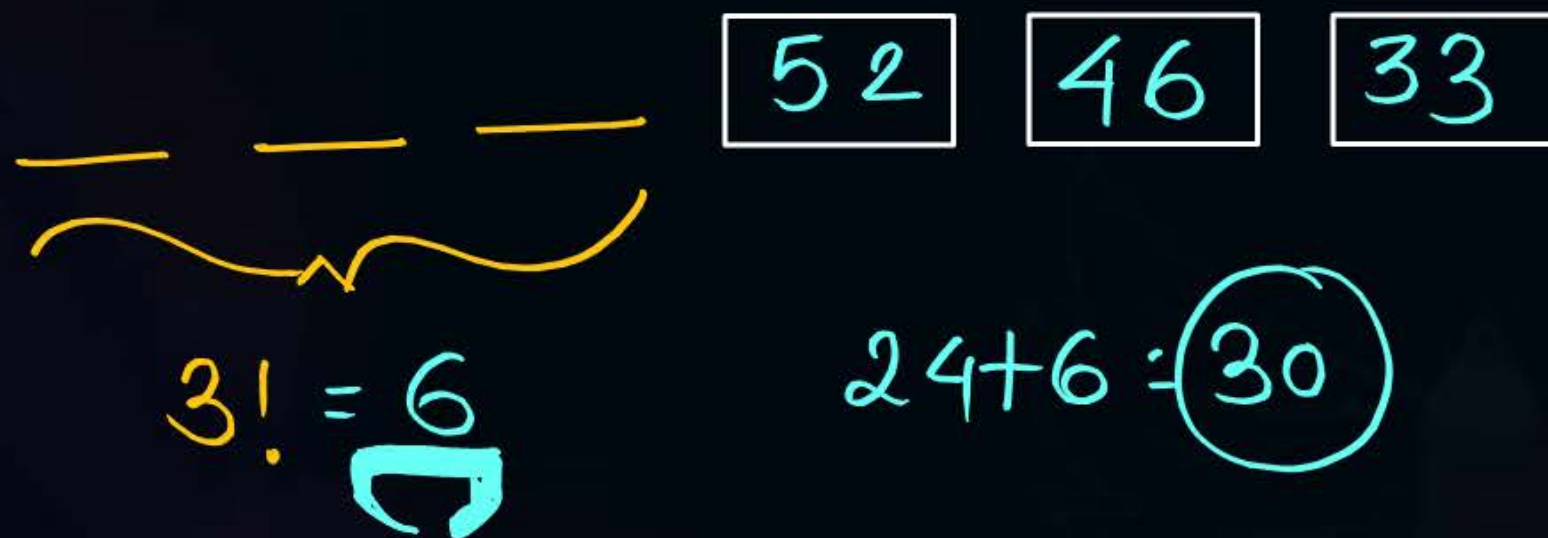


## Topic : Hashing

How many different insertion sequences of the key values using the same hash function and linear probing will result in the hash table shown above?

Counting

- (A) 10
- (B) 20
- (C) 30 ✓
- (D) 40



|   |    |
|---|----|
| 0 |    |
| 1 |    |
| 2 | 42 |
| 3 | 23 |
| 4 | 34 |
| 5 | 52 |
| 6 | 46 |
| 7 | 33 |
| 8 |    |
| 9 |    |



THANK - YOU

Double hashing  
Quadratic probing